AIO-Mesh: Hardware-Safe AIOps Orchestration for Cloud-Native Distributed Services over Programmable Infrastructures

Hanh P. Du[a], Hung D. Hoang[a], Thu H. Le[b], Tuan X. Le[b], Giang L. Nguyen[c], Hoa N. Nguyen[a,*]

[a] *Department of Information Systems, VNU University of Engineering and Technology, Hanoi, 11314, Vietnam*
[b] *National Data Center, Ministry of Public Security, Hanoi, 11309, Vietnam*
[c] *Institute of Information Technology, Vietnam Academy of Science and Technology, Hanoi, 11355, Vietnam*

Abstract

Future cloud-native distributed infrastructures increasingly combine Kubernetes-based lifecycle management, service-chain orchestration, software-defined control, and programmable data planes. In such environments, AIOps policies are expected to make online placement, steering, admission, and migration decisions from telemetry and runtime feedback. However, an AI-generated decision may be attractive in an optimization model while remaining physically non-deployable because of hard infrastructure constraints, such as compute capacity, memory capacity, and SRv6 Maximum Segment Depth (MSD) in programmable forwarding pipelines. This paper presents AIO-Mesh, a hardware-safe AIOps orchestration framework for cloud-native distributed services over programmable infrastructures. AIO-Mesh embeds a learned policy in a closed-loop runtime that connects telemetry-driven state construction, policy inference, safety validation, Kubernetes/Tekton-based VNF lifecycle control, SRv6/P4 steering, and data-plane feedback. At its core, HARP, a Hardware-Aware Reinforcement Policy, combines graph-attention topology encoding, MaskablePPO, hard action masking, adaptive operational penalties, and smart admission control to jointly optimize placement, routing, and proactive migration. The central design principle is to separate hard deployability constraints from soft operational objectives: CPU, memory, and MSD infeasibility are removed before policy sampling and admission, while latency, service-level pressure, and migration cost are optimized through the reward function. Experiments across Vietnam, NSFNET, and GEANT2 topologies show that HARP improves safe acceptance under stress workloads. A hardware-in-the-loop testbed with MicroK8s, Tekton, Mininet, BMv2 P4 switches, and SRv6 steering further demonstrates that AIO-Mesh maintains zero admitted MSD violations across normal, heavy-tail, topology-stress, DDoS make-before-break, and chaos scenarios.

Keywords: AIOps, Cloud-native distributed services, Hardware-safe orchestration, Service function chaining, Network function virtualization, Programmable infrastructures, SRv6, P4, Safe reinforcement learning, Runtime admission control

1. Introduction

Future cloud-native infrastructures are becoming distributed, programmable, and operationally dynamic. Modern service platforms combine Kubernetes clusters, edge and regional compute sites, software-defined networking, telemetry pipelines, workflow engines, and programmable forwarding devices. These infrastructures no longer host only conventional microservices; they increasingly host network and security services such as firewalls, intrusion detection systems, load balancers, deep packet inspection functions, gateways, and other virtualized or containerized network functions. In security-oriented service chains, these functions may also include AI-powered traffic inspection and intrusion-detection modules; prior studies on large-scale traffic sensing, real-time intrusion detection, and hybrid signature–deep analysis pipelines show that such AI-enabled security functions are increasingly deployable as operational traffic-processing services [1]. Such functions are composed into service chains to enforce traffic-processing policies across tenants, applications, and security domains. As a result, infrastructure operation requires continuous decisions about where to place service functions, how to steer traffic through them, when to migrate overloaded instances, and how to preserve service objectives under changing demand, heterogeneous resources, and topology-dependent latency.

This problem is naturally an AIOps problem rather than a one-shot optimization problem. In a production cloud-native environment, orchestration decisions are made from telemetry, policy intent, service demand, resource state, and runtime feedback. A learned policy must also be operated as a managed artifact: it must be loaded with the correct topology and normalization state, invoked through a stable inference path, monitored during execution, constrained by safety checks, and connected to cloud-native

*Corresponding author
Email addresses: hanhdp@vnu.edu.vn (Hanh P. Du), 22028293@vnu.edu.vn (Hung D. Hoang), lehthub1@gmail.com (Thu H. Le), tuanlx.psa@gmail.com (Tuan X. Le), nlgiang@ioit.ac.vn (Giang L. Nguyen), hoa.nguyen@vnu.edu.vn (Hoa N. Nguyen)

and network-control actuators. This view is consistent with recent MLOps and AIOps literature, which emphasizes that machine-learning models must move from isolated training pipelines into monitored, versioned, trustworthy, and feedback-driven operational loops [2, 3, 4, 5, 6, 7]. In infrastructure orchestration, however, the runtime faces an additional challenge: an AI decision may be valid in a logical model but non-deployable on the physical or programmable substrate.

Programmable data planes make this deployability gap explicit. Segment Routing over IPv6 (SRv6) allows an ingress node to encode a list of Segment Identifiers (SIDs) in the packet header, enabling stateless traffic steering through paths and service functions [8, 9, 10]. P4-programmable switches expose packet-processing behavior through programmable parser, match-action, and deparser stages [11]. Together, SRv6 and P4 provide a practical substrate for steering cloud-native service chains without maintaining per-flow state at every intermediate node. Yet this flexibility is bounded by hardware. A programmable parser can inspect only a limited number of SIDs in a single pass. This limit, commonly represented by the Maximum Segment Depth (MSD), is not a tunable preference but a hard parsing boundary. If a selected service path requires a SID stack deeper than the supported MSD of the relevant forwarding profile, the action is not merely suboptimal; it is not admissible.

Existing orchestration methods do not fully address this operational safety requirement. Classical ILP and heuristic formulations provide useful models for service-chain placement and routing, but they are often too expensive or too myopic for online operation. Recent deep reinforcement learning and graph-based methods improve adaptive SFC embedding and VNF placement [12, 13, 14, 15, 16], and migration-oriented studies address failure prediction, load pressure, and migration budgets [17, 18, 19, 20, 21, 22]. SRv6-aware approaches such as SFCPlanner show the relevance of source-routed flow steering for service-chain planning [23]. Nevertheless, many learned and optimization-based approaches still treat the data plane as an ideal graph, handle parser limits as post-hoc checks, or encode non-deployability as a soft reward penalty. This is insufficient for AIOps control, where unsafe actuation can convert a policy error into a data-plane failure.

The core research question addressed in this paper is therefore: *how can an AIOps runtime use learned policies for cloud-native distributed service orchestration while ensuring that every admitted action remains deployable on a programmable infrastructure?* This question has three coupled challenges. First, the decision policy must be topology-aware because placement, routing, SID depth, and latency depend on the structure of the substrate graph. Second, the runtime must distinguish hard deployability constraints from soft operational objectives. CPU, memory, and MSD feasibility must be enforced before actuation, whereas latency, service-level pressure, and migration overhead can be optimized through the reward function. Third, the learned policy must be embedded into a closed-loop AIOps runtime that connects telemetry, state construction, inference, safety validation, cloud-native lifecycle management, programmable steering, and feedback monitoring.

To address these challenges, we propose **AIO-Mesh**, a hardware-safe AIOps orchestration framework for cloud-native distributed services over programmable infrastructures. AIO-Mesh treats the learned policy as one component of an operational control loop rather than as a standalone optimizer. Its runtime integrates telemetry-driven state construction, policy inference, hard-feasibility validation, smart admission control, Kubernetes/Tekton-based VNF lifecycle actions, SRv6/P4 steering, and data-plane feedback. At the core of AIO-Mesh, we introduce **HARP** (**H**ardware-**A**ware **R**einforcement **P**olicy), a graph-reinforcement policy for joint placement, routing, and proactive migration. HARP combines graph-attention topology encoding, MaskablePPO, invalid action masking, adaptive penalties for soft service objectives, and explicit `NO_SAFE_ACTION` handling under saturation.

The main contributions of this paper are summarized as follows.

- We formulate hardware-safe AIOps orchestration for cloud-native distributed service chains over programmable infrastructures, explicitly separating hard deployability constraints from soft operational objectives. In this formulation, SRv6 MSD feasibility is treated as an admission condition rather than a reward penalty.
- We design **AIO-Mesh**, a closed-loop AIOps runtime that operationalizes a learned policy through telemetry ingestion, normalized state construction, inference serving, safety validation, cloud-native lifecycle management, programmable SRv6/P4 actuation, and runtime feedback.
- We propose **HARP**, a hardware-aware reinforcement policy that combines graph-attention topology encoding, masked policy optimization, adaptive operational penalties, and smart admission control to jointly optimize service placement, traffic steering, and proactive migration.
- We introduce a hardware-safe action-masking and admission mechanism that removes CPU-, memory-, and MSD-infeasible actions before sampling or actuation, and returns explicit `NO_SAFE_ACTION` outcomes when the feasible set is exhausted.
- We evaluate AIO-Mesh through multi-topology stress simulations and a hardware-in-the-loop cloud-native testbed integrating MicroK8s, Tekton, Mininet, BMv2 P4 switches, SRv6 steering, and telemetry feedback. The evaluation distinguishes pre-admission unsafe candidates from admitted data-plane violations, which is essential for interpreting safety-preserving rejection under saturation.

2. Background and Related Work

This section positions AIO-Mesh at the intersection of AIOps, cloud-native resource orchestration, service function chaining, programmable data planes, and safe reinforcement learning. The key distinction is that AIO-Mesh is not only a placement or routing algorithm. It is an AIOps runtime in which a learned policy must be observed, constrained, admitted, actuated, and verified before it changes the state of a programmable distributed infrastructure.

2.1. AIOps and MLOps for Cloud-Native Infrastructure

AIOps applies AI techniques to the operation of complex software and infrastructure systems. Typical tasks include anomaly detection, failure prediction, incident triage, capacity planning, root-cause analysis, and automated remediation. Recent AIOps and MLOps surveys emphasize that modern operational systems rely on heterogeneous telemetry and increasingly require closed-loop workflows rather than isolated predictive models [3, 6]. MLOps complements this view by addressing the lifecycle of machine-learning artifacts, including model packaging, versioning, deployment, inference serving, monitoring, robustness, and rollback [2, 4, 5].

For cloud-native infrastructures, this operational perspective is essential. Placement, migration, scaling, and steering decisions are not made once; they are continuously revised as traffic demand, resource pressure, and service objectives change. Recent work on intelligent DevOps automation, Kubernetes-oriented microservice placement, elastic container management, dynamic cloud resource allocation, and computing-continuum resource selection similarly argues that automation systems must connect models, runtime evidence, and deployment actions [7, 24, 25, 26, 27, 28, 29]. AIO-Mesh follows this direction but focuses on a harder class of actuation: the policy output directly affects VNF lifecycle, traffic steering, and programmable data-plane behavior. Therefore, the runtime must be fail-closed and safety-aware, not merely prediction-driven.

2.2. Learning-Based Resource and Service-Chain Orchestration

Service Function Chaining (SFC) steers traffic through an ordered sequence of network functions, such as firewalls, intrusion detection systems, deep packet inspection functions, load balancers, and gateways [37]. In NFV-enabled and cloud-native infrastructures, these functions are implemented as VNFs or containerized network functions that can be instantiated, scaled, migrated, and removed by software controllers. The associated orchestration problem couples placement, routing, admission, and migration: a placement decision consumes compute and memory resources, a routing decision determines latency and path depth, and a migration decision changes both the service path and the residual resource distribution. Resource allocation and VNF embedding have also been investigated in network-slicing environments, where substrate resources, virtual links, and service-level constraints must be optimized jointly [38]. This line of work is complementary to AIO-Mesh: while prior VNF-embedding studies optimize logical resource allocation, AIO-Mesh focuses on embedding such decisions into a hardware-safe AIOps runtime with explicit programmable-data-plane admission semantics.

Classical ILP and heuristic formulations expose the structure of SFC constraints but are often unsuitable for online operation under dynamic traffic. Learning-based methods have therefore become increasingly common. DRL-SFCP formulates adaptive SFC placement as a reinforcement-learning problem [12]; GCN-based and graph-assisted approaches exploit topology information for SFC embedding and virtualized network embedding [13, 31, 14]; QoS-aware and reliable deployment methods address service-level objectives and robustness [15, 30, 39]; and recent work considers fast joint mapping for SFC deployment [16]. Migration-oriented studies further address failure prediction, AI-assisted 5G/B5G migration, long-term migration budgets, cluster migration, and reinforcement-learning-based migration planning [17, 18, 19, 20, 21, 22].

These methods improve adaptivity, but many still treat the data plane as an ideal forwarding graph. Feasibility is usually dominated by compute, memory, bandwidth, or latency. Hardware parser limits, header-stack depth, and target-specific programmable pipeline constraints are either ignored, treated as soft costs, or checked only after policy selection. This abstraction is problematic for AIOps orchestration because the policy is not producing recommendations; it is producing infrastructure actions.

2.3. Programmable Data Planes and Hardware-Constrained Actuation

SRv6 enables source-routed forwarding by encoding a list of SIDs in the IPv6 Segment Routing Header [8, 9, 10]. Each SID can represent a topological segment, a local behavior, or a service-processing instruction, which makes SRv6 attractive for service-chain steering without per-flow state in every intermediate device. P4 complements this model by making parser, match-action, and deparser behavior programmable [11]. Prior work has demonstrated P4-based service chaining, SRv6 processing, runtime programmability, test generation, in-switch caching, and stateful service-function deployment [32, 33, 40, 41, 42, 36, 35].

Programmable infrastructures, however, expose hard actuation constraints. A P4 pipeline is constrained by parser states, header stacks, match-action resources, recirculation behavior, and target-specific limits. For SRv6 service chaining, a visible constraint is the Maximum Segment Depth (MSD), i.e., the maximum SID depth that the forwarding profile can parse or process. A route that is reachable and low-latency at the graph level can still be non-deployable if the resulting SID stack exceeds MSD. SRv6-aware planning methods such as SFCPlanner and SRv6-INT-driven runtime adjustment show the importance

Table 1: Positioning of AIO-Mesh against representative related work.

Work line	**Primary focus**	**Cloud-native lifecycle**	**Graph-aware policy**	**Hard deployability safety**	**Programmable actuation**	**AIOps runtime**
Classical ILP / greedy heuristics	SFC placement and routing	No	Limited	No	No	No
DRL-based SFC placement [12, 15, 30]	Online SFC embedding	No	Partial	No	No	No
GCN/graph-assisted SFC/VNE [13, 31, 14]	Topology-aware embedding	No	Yes	No	No	No
SFC migration studies [17, 18, 19, 20, 21, 22]	Load/failure migration	Partial	Varies	No	No	Limited
SRv6/P4 service-chain systems [32, 33, 34, 35, 36]	Programmable steering	Limited	Partial	Partial	Yes	Limited
SFCPlanner [23]	SRv6-aware planning	No	Partial	Partial	Yes	No
AIOps/MLOps automation [2, 3, 4, 5, 6, 7]	Operational ML loops	Partial	Not network-specific	No	No	Yes
AIO-Mesh / HARP	Hardware-safe AIOps orchestration	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

of source-routed steering and telemetry-assisted reconfiguration [23, 34]. AIO-Mesh builds on this line of work, but shifts the focus from path computation to runtime safety: a learned AIOps policy must be constrained before actuation, and unsafe saturation must result in explicit rejection rather than best-effort deployment.

2.4. Safe and Graph-Aware Reinforcement Learning for Infrastructure Control

Reinforcement learning is attractive for infrastructure orchestration because each accepted request changes the residual capacity and future feasible action set. PPO is widely used because it balances training stability and policy improvement [43]. Graph neural networks, particularly Graph Attention Networks (GATs), are suitable for networked systems because they preserve topological structure that is lost in flat state representations [44]. HARP adopts this graph-aware view because placement quality depends on source–destination distance, candidate service locations, path length, SID-stack depth, and residual headroom.

The main safety issue is constraint semantics. Reward penalties are appropriate for soft operational objectives, such as latency, SLA pressure, load balance, and migration overhead. They are not sufficient for constraints that define binary deployability. Safe reinforcement learning and constrained policy optimization recognize the need to separate reward optimization from safety enforcement [45, 46]. Invalid action masking provides a practical mechanism for discrete or structured action spaces by removing invalid actions before sampling [47]. In AIO-Mesh, this mechanism is used not only to improve exploration but to enforce a hardware-safety contract: CPU-, memory-, and MSD-infeasible actions are excluded before they can become admitted infrastructure actions.

2.5. Research Gap Summary

The literature shows substantial progress in cloud-native resource management, SFC placement, learning-based orchestration, programmable data-plane steering, and AIOps/MLOps automation. However, these streams remain only partially connected. Existing orchestration algorithms often optimize logical placement and routing without guaranteeing that the resulting SRv6/P4 action is parser-feasible. Learning-based methods improve adaptivity but often rely on reward penalties for constraints that should be enforced as hard admission conditions. Programmable data-plane systems demonstrate how to steer packets, but they rarely define how an AI policy should be guarded before actuation. AIOps/MLOps research provides principles for operational loops and model lifecycle management, but it rarely addresses the hardware-safety semantics of programmable network-service actuation.

Table 1 summarizes the positioning of AIO-Mesh. The key gap is the absence of a unified framework that combines cloud-native lifecycle management, graph-aware policy learning, hard hardware-safety enforcement, smart admission, programmable actuation, and runtime validation.

Based on this gap, AIO-Mesh is designed around four principles. First, learned orchestration must be topology-aware because service-chain feasibility depends on the substrate graph. Second, hard deployability constraints must be enforced before actuation; MSD violations cannot be delegated to a soft penalty. Third, unsafe saturation should result in explicit admission rejection, not in best-effort deployment. Fourth, the learned policy must be embedded in an AIOps runtime that connects telemetry, inference, safety validation, lifecycle management, programmable steering, and feedback. These principles motivate the system model and design requirements defined in the next section.

3. System Model and Design Requirements

This section defines the substrate, service-chain, and hardware-safety models used by AIO-Mesh. The purpose is to separate two notions that are often conflated in learning-based orchestration: an action may be attractive under an optimization objective, but still be non-deployable on the cloud-native and programmable-data-plane substrate. We therefore model the infrastructure state, the service-chain decision, the SRv6/P4 deployability constraints, and the admission semantics that the AIOps runtime must enforce before actuation.

3.1. Cloud-Native Programmable Substrate

We model the infrastructure as a distributed substrate graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ represents a cloud, edge, or regional NFVI site capable of hosting virtualized or containerized network functions, and each edge $e = (u, v) \in \mathcal{E}$ represents a physical or logical inter-site link. Each node has compute and memory capacities $\mathbf{c}_v = [c_v^{cpu}, c_v^{mem}]$, and is associated with a programmable-data-plane profile whose SRv6 Maximum Segment Depth (MSD) is denoted by m_v^{msd}. The MSD is not a consumable resource. It is a parser-depth bound that constrains whether a candidate SID stack can be processed by the corresponding SRv6/P4 forwarding profile. Each edge $e = (u, v)$ has a latency l_{uv}. Additional link-level attributes, such as bandwidth or loss, may be enforced by the controller, but are not included in the core policy interface in order to keep the deployed observation contract stable.

At decision epoch t, the residual compute state of node v is $\mathbf{r}_v(t) = [r_v^{cpu}(t), r_v^{mem}(t)]$. AIO-Mesh observes the substrate through telemetry and converts it into a policy-facing state. For a substrate with $N = |\mathcal{V}|$ nodes, the HARP state is $s_t = (\mathbf{X}_t, \mathbf{g}_t, \mathcal{A}_{topo})$, where $\mathbf{X}_t \in \mathbb{R}^{N\times 6}$ is the node-feature matrix, $\mathbf{g}_t \in \mathbb{R}^{13}$ contains global request and operational features, and $\mathcal{A}_{topo}$ is the substrate adjacency matrix. The node features encode resource pressure, MSD-related headroom or warning indicators, alert state, and topology-related context. This representation keeps the inference interface fixed while allowing the graph encoder to exploit the physical substrate topology.

3.2. Service-Chain Request and Orchestration Decision

An incoming service-chain request is represented as

$$q = (o_q, z_q, \mathcal{F}_q, \mathbf{d}_q, \tau_q, \kappa_q), \tag{1}$$

where o_q and z_q are the traffic origin and destination, $\mathcal{F}_q = [f_1, f_2, \ldots, f_m]$ is the ordered service-function chain, $\mathbf{d}_q$ denotes the per-function resource demand, τ_q is the service-level latency target, and κ_q is the service class or priority. For each function f_i, the compute and memory demands are denoted by $d_{q,i}^{cpu}$ and $d_{q,i}^{mem}$.

A placement decision maps service functions to substrate nodes: $\phi_q : \mathcal{F}_q \to \mathcal{V}$, where $\phi_q(f_i) = v$ means that f_i is instantiated or reused at node v. Given a placement, the routing decision selects substrate paths between the origin, consecutive service-function locations, and the destination. The resulting end-to-end service path is

$$P_q = P_{o_q,f_1} \oplus P_{f_1,f_2} \oplus \cdots \oplus P_{f_m,z_q}. \tag{2}$$

The orchestration action therefore couples placement, routing, and operational mode: $a_t = (\phi_q, P_q, m_t)$, where m_t indicates whether the runtime admits a new chain, reuses an existing instance, scales out, or migrates one or more functions.

Migration is modeled as a controlled transition from an old placement ϕ_q^{old} to a new placement ϕ_q^{new}. Its operational cost is

$$C_{mig}(q, t) = \sum_{f_i \in \mathcal{F}_q} \mathbb{I}\big[\phi_q^{old}(f_i) \neq \phi_q^{new}(f_i)\big]\, \mu_i, \tag{3}$$

where μ_i summarizes deployment delay, state-transfer cost when applicable, and steering-update overhead. AIO-Mesh executes migration using a make-before-break discipline: the replacement function is created and verified before traffic is steered to the new path, and the old instance is removed only after steering confirmation. This paper treats this as a safety-preserving control sequence rather than as a zero-downtime guarantee, which would require packet-level continuity evidence.

3.3. Hardware-Safety and Admission Constraints

For a candidate action a_t, the SRv6/P4 controller translates the selected service path into a SID list

$$\mathcal{S}_q(a_t) = \big[SID_1, SID_2, \ldots, SID_{h_q(a_t)}\big], \tag{4}$$

where $h_q(a_t) = |\mathcal{S}_q(a_t)|$ is the induced SID-stack depth. The action is data-plane deployable only if the resulting stack can be parsed by all relevant ingress or forwarding profiles:

$$h_q(a_t) \le m_v^{msd}, \quad \forall v \in \mathcal{P}_q^{parse}(a_t), \tag{5}$$

where $\mathcal{P}_q^{parse}(a_t)$ denotes the set of nodes or profiles that must parse the SID stack. Equation (5) is a hard deployability constraint. It is not a latency penalty or a degraded-service condition: if the stack exceeds the supported parser depth, the action must not be admitted.

Compute and memory feasibility are also enforced before admission:

$$\sum_{q \in \mathcal{Q}_t} \sum_{f_i \in \mathcal{F}_q} \mathbb{I}[\phi_q(f_i) = v]\, d_{q,i}^{cpu} \le c_v^{cpu}, \quad \forall v \in \mathcal{V}, \tag{6}$$

$$\sum_{q \in \mathcal{Q}_t} \sum_{f_i \in \mathcal{F}_q} \mathbb{I}[\phi_q(f_i) = v]\, d_{q,i}^{mem} \le c_v^{mem}, \quad \forall v \in \mathcal{V}. \tag{7}$$

Here, $\mathcal{Q}_t$ is the set of active and newly considered requests at epoch t. Constraints (5)–(7) define hard feasibility. They are enforced by action masking and runtime admission control, not only by reward shaping.

This separation gives each candidate action one of three operational outcomes. An action is *admissible* if it satisfies compute, memory, and MSD feasibility and can be passed to cloud-native and data-plane actuators. It is an *unsafe candidate* if it is generated or considered internally but rejected before admission because at least one hard constraint fails. It is an *admitted violation* if it is accepted by the orchestrator and later produces a data-plane violation, such as an MSD drop. A hardware-safe orchestrator may observe unsafe candidates under saturation, but it should prevent them from becoming admitted violations.

3.4. Operational Objective

AIO-Mesh aims to admit as much useful service demand as possible while preserving hard deployability and controlling operational cost. Let $A_q(t) \in \{0,1\}$ indicate whether request q is admitted at epoch t, $L_q(t)$ denote its service latency, $C_{mig}(q,t)$ denote migration cost, and $V_q^{sla}(t)$ denote normalized service-level violation. The generic utility objective is

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t}\sum_{q\in\mathcal{Q}_t} A_q(t)\Big(\alpha - \beta L_q(t) - \gamma C_{mig}(q,t) - \delta V_q^{sla}(t)\Big)\right] \tag{8}$$

subject to the hard feasibility constraints in Eqs. (5)–(7). The coefficients α, β, γ, and δ control the relative importance of admission, latency, migration cost, and SLA pressure. Equation (8) should be read as an operational objective rather than as an offline solver formulation: HARP learns to rank feasible actions, while AIO-Mesh enforces hard feasibility before actuation.

Accordingly, the evaluation distinguishes safe acceptance from unsafe candidate pressure and admitted MSD violations. Safe acceptance measures useful admitted demand under hard feasibility. Unsafe candidate rate measures how often the workload pushes the system outside the feasible region before admission. Admitted MSD violation rate measures actual data-plane safety failure after admission. This distinction is essential for stress scenarios: rejecting an unsafe request may reduce raw acceptance, but it is the correct behavior if all remaining actions are non-deployable.

3.5. Design Requirements

The above model leads to five requirements for AIO-Mesh. **R1: hardware safety by construction**. CPU, memory, and MSD feasibility must be checked before actuation; the policy must not rely on data-plane drops to discover unsafe actions. **R2: topology-aware decision making**. Placement, routing, latency, and SID depth depend on the substrate graph, motivating the graph-attention encoder used by HARP. **R3: closed-loop observability**. The orchestrator must consume telemetry and expose decisions, admission rejections, steering status, and data-plane counters for subsequent control. **R4: cloud-native lifecycle integration**. Admitted decisions must be translated into concrete deployment, readiness checking, migration, and steering operations. **R5: safe rejection under saturation**. When no feasible action exists, the correct outcome is `NO_SAFE_ACTION`, not best-effort deployment of an unsafe service path.

These requirements define the boundary between the learning policy and the operational runtime. HARP optimizes over topology-aware and hardware-feasible actions, while AIO-Mesh remains responsible for fail-closed admission, lifecycle actuation, programmable steering, and feedback validation.

4. AIO-Mesh Architecture

AIO-Mesh is a closed-loop AIOps orchestration framework that connects a learned policy with cloud-native lifecycle control and programmable data-plane actuation. Its architectural principle is simple: the policy may propose placement, routing, or migration actions, but no action is allowed to reach the infrastructure until the runtime has validated compute, memory, and SRv6 MSD feasibility. This separates policy optimization from operational admission and prevents a high-scoring but non-deployable decision from being installed in the data plane.

Figure 1 shows the proposed architecture. AIO-Mesh consists of five tightly coupled functions: telemetry-driven state construction, policy inference, safety and admission control, cloud-native lifecycle actuation, and SRv6/P4 steering with feedback. HARP is the learned decision engine inside this loop. The surrounding runtime provides the operational guarantees required to use HARP safely in a programmable cloud-native environment.

4.1. Closed-Loop Runtime Organization

The AIO-Mesh runtime follows an observe–decide–admit–actuate–verify loop. The telemetry layer collects resource, service, orchestration, and data-plane signals, including CPU and memory pressure, active VNF placement, service-chain demand, alert state, SRv6 steering status, and MSD-related counters. The state manager normalizes these signals and constructs the policy-facing graph state defined in Section 3. This state is passed to the policy runtime together with the substrate topology.

The policy runtime hosts two decision paths. Under stable load, AIO-Mesh may use a deterministic heuristic path to reduce overhead. Under stress, alert, or migration pressure, it invokes HARP to produce a topology-aware placement–routing–migration action. The heuristic and learned paths share the same safety boundary; neither path is allowed to actuate directly. This design makes the learned policy replaceable or retrainable without changing the admission contract of the system.

The admission layer is the boundary between decision making and infrastructure modification. It validates the

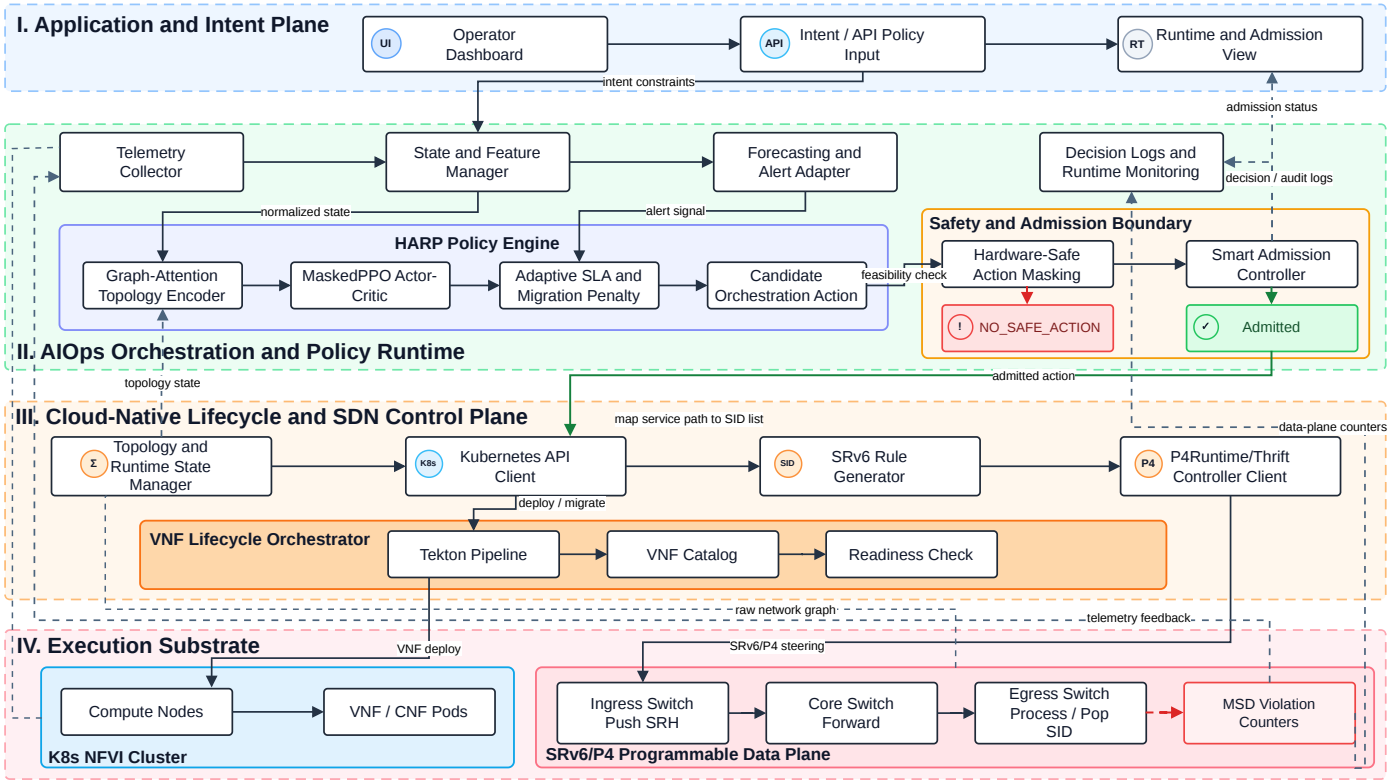


Figure 1: AIO-Mesh closed-loop AIOps runtime for hardware-safe orchestration of cloud-native distributed services over programmable infrastructures. HARP generates candidate orchestration actions from telemetry-derived state, while the safety and admission boundary enforces CPU, memory, and SRv6 MSD feasibility before cloud-native lifecycle operations and SRv6/P4 steering are applied.

candidate action against residual compute and memory capacity, derives the SID stack associated with the selected service path, and checks whether the stack satisfies the MSD of the relevant programmable forwarding profile. A feasible action is admitted and passed to the actuation layer. An infeasible action is rejected before deployment or steering, and if no feasible action remains, the runtime returns `NO_SAFE_ACTION`.

The actuation layer translates admitted decisions into cloud-native and network-control operations. Kubernetes and workflow automation are used to deploy, reuse, scale, or migrate VNFs. The SRv6/P4 interface generates steering state, installs the corresponding forwarding rules, and reports steering confirmation or data-plane anomalies. These post-actuation signals are fed back to the telemetry layer, closing the AIOps loop.

4.2. Policy Runtime and State Management

AIO-Mesh treats HARP as an operational policy artifact rather than as an isolated model file. The runtime therefore manages the trained policy, topology descriptor, normalization state, safety configuration, and controller endpoints as a consistent deployment unit. This is necessary because an otherwise valid policy can become unsafe if it is executed with stale topology information, inconsistent preprocessing, or an incompatible data-plane profile.

The state manager maintains the deployed observation contract. It maps heterogeneous telemetry into a fixed graph-structured state containing node features, global request features, and the substrate adjacency matrix. The same contract is used for HARP inference and for runtime admission logging. For each decision, AIO-Mesh records the request context, selected action, feasibility outcome, rejection reason if applicable, and post-actuation feedback. These logs allow operators to distinguish policy unavailability, lack of feasible actions, controller failure, and data-plane violation.

This runtime organization follows an MLOps discipline but is specialized for network operations. Model versioning, preprocessing consistency, and observability are not sufficient by themselves; the policy also needs a fail-closed safety boundary because its output changes infrastructure state. AIO-Mesh therefore does not silently push a learned decision when the model, topology descriptor, normalization state, or controller endpoint is unavailable or inconsistent. In such cases, the runtime either falls back to a verified safety-preserving heuristic path or returns an explicit rejection.

4.3. Safety Boundary and Admission Semantics

The safety boundary enforces the distinction between candidate decisions and admitted infrastructure actions. A candidate action may be generated by HARP or by the

heuristic path, but it becomes operational only after passing the hard feasibility checks. CPU and memory feasibility are checked against residual node capacity. SRv6 feasibility is checked by translating the selected service path into a SID stack and comparing its depth with the MSD supported by the relevant programmable forwarding profile.

This design gives the runtime fail-closed behavior. If the feasible action set is empty, the result is `NO_SAFE_ACTION`, not best-effort deployment. If steering confirmation is not received, the previous service path is preserved. If a replacement VNF cannot be verified during migration, the old instance is not removed. If an unexpected MSD-drop counter is observed after actuation, the decision context is logged and subsequent requests can be forced through the conservative admission path until the controller state and topology metadata are reconciled.

The distinction between an unsafe candidate and an admitted violation is central to AIO-Mesh. Under overload, many candidate decisions may be infeasible because the infrastructure has exhausted safe compute or SID-depth headroom. This is not itself a safety failure. The failure occurs only when such a candidate is admitted and installed in the data plane. AIO-Mesh is therefore designed to convert unsafe candidates into explicit admission rejections before they become data-plane behavior.

4.4. Cloud-Native Lifecycle Actuation

For admitted actions, AIO-Mesh invokes cloud-native lifecycle operations. A new service-chain request may require creating one or more VNFs, reusing existing instances, exposing service endpoints, and preparing steering state. A migration action follows a stricter make-before-break sequence. The replacement VNF is created first, Kubernetes readiness is verified, the new endpoint is translated into an SRv6-safe service path, and the controller installs the new steering state. Only after steering confirmation can the old instance be removed.

This separation keeps responsibilities clear. The policy selects an admissible orchestration intent; the lifecycle layer executes deployment and migration workflows; and the SRv6/P4 controller realizes the forwarding behavior. The architecture does not rely on an unqualified zero-downtime claim. Make-before-break sequencing is a safety-preserving control mechanism, while strict zero-downtime migration would require packet-level evidence such as TCP continuity or packet-capture validation.

4.5. Programmable Data-Plane Interface

The programmable data-plane interface maps admitted service-chain actions to SRv6/P4 forwarding behavior. Given the selected placement and routing descriptor, the controller derives the concrete service path, generates the SID list, installs the corresponding steering state, and monitors forwarding counters. The data plane is therefore both an actuator and a verifier: it applies admitted SRv6 paths and reports whether steering has been installed consistently or whether MSD-related drops occur.

Algorithm 1 AIO-Mesh request lifecycle

Input: SFC request q, substrate graph $\mathcal{G}$, telemetry T_t
Output: Admission outcome and optional actuation result

$s_t \leftarrow \textsc{BuildState}(q, \mathcal{G}, T_t)$
if $\textsc{IsStable}(T_t)$ **then**
  $a_t \leftarrow \textsc{HeuristicDecision}(s_t)$
else
  $a_t \leftarrow \textsc{HARPDecision}(s_t, \mathcal{G})$
end if
$(ok, reason) \leftarrow \textsc{ValidateHardConstraints}(a_t, q, \mathcal{G})$
if $ok =$ **false then**
  return `NO_SAFE_ACTION`$(reason)$
end if
if $\textsc{RequiresMigration}(a_t)$ **then**
  $\textsc{CreateReplacementVNF}(a_t)$
  $\textsc{WaitUntilReady}(a_t)$
  $\textsc{InstallSRv6Steering}(a_t)$
  $\textsc{RemoveOldVNFAfterConfirmation}(a_t)$
else
  $\textsc{DeployOrReuseVNF}(a_t)$
  $\textsc{InstallSRv6Steering}(a_t)$
end if
$\textsc{CollectFeedback}()$
return `ADMITTED`

AIO-Mesh deliberately avoids relying on post-hoc repair, such as Binding SID compression, to make unsafe paths deployable after policy selection. Such mechanisms can be useful in operational networks, but they weaken the learning and admission semantics. In this work, an action must be directly admissible under the configured SRv6/P4 profile before actuation. HARP selects a compact orchestration action, while the controller realizes it through a concrete feasible path and SID list. This keeps the policy action space tractable while preserving data-plane deployability through controller-side realization and runtime admission validation.

4.6. End-to-End Request Lifecycle

Algorithm 1 summarizes the lifecycle of a service-chain request. The runtime first constructs the telemetry-derived state, selects a candidate action using either the heuristic path or HARP, validates hard constraints, and then actuates only admitted decisions. Rejected requests return `NO_SAFE_ACTION` with a reason. Accepted requests trigger VNF deployment, reuse, or make-before-break migration, followed by SRv6/P4 steering and feedback collection.

This lifecycle defines the operational contract used by the rest of the paper. HARP is responsible for selecting high-quality topology-aware actions. AIO-Mesh is responsible for fail-closed admission, lifecycle execution, programmable steering, and feedback validation. The next section details how HARP constructs and optimizes the hardware-aware policy used inside this runtime.

5. HARP: Hardware-Aware Reinforcement Policy

HARP is the learned decision engine inside AIO-Mesh. Its objective is to select high-quality orchestration actions while preserving the hard deployability constraints defined in Section 3. The key design principle is to separate *hard feasibility* from *soft optimization*. CPU, memory, and SRv6 MSD feasibility are enforced through action masking and admission control before actuation. Latency, SLA pressure, load balance, and migration overhead are optimized through the reward function. HARP therefore does not learn to "tolerate" data-plane-infeasible actions; it removes them from the admissible action space before sampling.

HARP combines three mechanisms: a graph-attention encoder for topology-aware state representation, masked policy optimization for hard constraints, and adaptive penalties for soft operational objectives. The selected action is not applied directly to the infrastructure. It is returned to the AIO-Mesh runtime, which performs final admission validation, lifecycle actuation, and SRv6/P4 steering.

5.1. MDP Formulation

We formulate online service-chain orchestration as a Markov Decision Process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma), \tag{9}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ denotes the transition dynamics induced by traffic arrivals and infrastructure updates, $\mathcal{R}$ is the reward function, and γ is the discount factor.

At decision epoch t, the state is

$$s_t = (\mathbf{X}_t, \mathbf{g}_t, \mathcal{A}_{topo}), \tag{10}$$

where $\mathbf{X}_t \in \mathbb{R}^{N\times 6}$ is the node-feature matrix, $\mathbf{g}_t \in \mathbb{R}^{13}$ contains request-level and global operational features, and $\mathcal{A}_{topo}$ is the substrate adjacency matrix. This state representation follows the system model in Section 3: runtime dynamics are encoded in the node and global features, while the physical topology is preserved as a graph prior.

The abstract orchestration action in Section 3 couples placement, routing, and operational mode. In the implemented policy, HARP uses the compact parameterization

$$a_t = (v_t^{place}, v_t^{anchor}, m_t), \tag{11}$$

where $v_t^{place} \in \mathcal{V}$ is the target node for placement or migration, $v_t^{anchor} \in \mathcal{V}$ is a routing anchor used by the controller to derive a concrete service path, and $m_t \in \mathcal{M}$ is the operation mode, such as admission, reuse, scale-out, or migration. The action-space size is therefore

$$|\mathcal{A}| = |\mathcal{V}| \times |\mathcal{V}| \times |\mathcal{M}|. \tag{12}$$

HARP does not enumerate packet-level forwarding rules. Instead, for each compact action, AIO-Mesh derives candidate service paths from a cached K-shortest-path table and recomputes feasibility using the current request demand, residual compute and memory, and SRv6 MSD profile. This keeps the policy action space tractable while ensuring that each admitted action has at least one concrete deployable realization.

5.2. Graph-Attention State Encoder

A flat state encoder is insufficient for service-chain orchestration because feasibility depends on topological relations among source, destination, candidate service locations, and routing anchors. HARP therefore uses a graph-attention encoder to represent the substrate topology and runtime node state [44].

Let $\mathbf{H}^{(0)} = \mathbf{X}_t$. For attention head k, the unnormalized attention coefficient from node j to node i is

$$e_{ij}^{(k)} = \mathrm{LeakyReLU}\left(\mathbf{a}_k^\top [\mathbf{W}_k \mathbf{h}_i \| \mathbf{W}_k \mathbf{h}_j]\right), \quad j \in \mathcal{N}(i), \tag{13}$$

and the normalized attention weight is

$$\alpha_{ij}^{(k)} = \frac{\exp(e_{ij}^{(k)})}{\sum_{r\in\mathcal{N}(i)} \exp(e_{ir}^{(k)})}. \tag{14}$$

The updated node representation is

$$\mathbf{h}_i' = \Big\|_{k=1}^{K} \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}_k \mathbf{h}_j\right), \tag{15}$$

where K is the number of attention heads and $\sigma(\cdot)$ is a nonlinear activation. After L graph-attention layers, the graph embedding is pooled and concatenated with global request features:

$$\mathbf{z}_t = [\mathrm{Pool}(\mathbf{H}^{(L)}) \| \mathbf{g}_t]. \tag{16}$$

The vector $\mathbf{z}_t$ is passed to the actor and critic heads. This design allows the policy to reason about residual capacity, source–destination context, path length, SID-depth pressure, and future placement headroom in a topology-aware manner.

5.3. Hard-Feasibility Action Mask

The central safety mechanism in HARP is the hard-feasibility mask. Let $\mathcal{F}(s_t) \subseteq \mathcal{A}$ denote the feasible action set at state s_t:

$$\mathcal{F}(s_t) = \{a \in \mathcal{A} \,|C_{cpu}(a, s_t) = 0,\ C_{mem}(a, s_t) = 0,\ C_{msd}(a, s_t) = 0\}. \tag{17}$$

Here C_{cpu}, C_{mem}, and C_{msd} are hard-infeasibility indicators. In particular,

$$C_{msd}(a, s_t) = 1 \Longleftrightarrow h_q(a) > m_v^{msd} \tag{18}$$

for at least one relevant parser profile, where $h_q(a)$ is the SID-stack depth induced by action a, and m_v^{msd} is the supported SRv6 MSD of profile v.

The feasible set is constructed by filtering candidate placement nodes by residual CPU and memory, deriving candidate service paths for each remaining placement–anchor pair, translating each path into a SID list, and removing actions whose SID depth exceeds the relevant MSD profile. If multiple feasible paths remain for the same compact action, the controller selects the lowest-cost feasible path. With cached candidate paths and SID-depth metadata, mask construction reduces to checking current resource demand and header-depth feasibility over the candidate action set.

The actor produces a logit z_a for each action $a \in \mathcal{A}$. HARP applies the binary mask

$$M_t(a) = \begin{cases} 1, & a \in \mathcal{F}(s_t), \\ 0, & a \notin \mathcal{F}(s_t), \end{cases} \tag{19}$$

and replaces infeasible logits by $-\infty$:

$$\tilde{z}_a = \begin{cases} z_a, & M_t(a) = 1, \\ -\infty, & M_t(a) = 0. \end{cases} \tag{20}$$

The masked policy is then

$$\pi_\theta^M(a \mid s_t) = \frac{\exp(\tilde{z}_a)}{\sum_{b \in \mathcal{A}} \exp(\tilde{z}_b)}. \tag{21}$$

HARP optimizes this masked policy using PPO [43]. With

$$\rho_t(\theta) = \frac{\pi_\theta^M(a_t \mid s_t)}{\pi_{\theta_{old}}^M(a_t \mid s_t)}, \tag{22}$$

the clipped objective is

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta)\hat{A}_t, \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{23}$$

with standard value and entropy terms added during training. This masking mechanism is related to invalid action masking in policy-gradient methods [47], but its role here is stronger: it enforces a deployability boundary before action sampling.

5.4. Reward for Soft Operational Objectives

HARP uses the reward function only for objectives that admit trade-offs. Latency, migration cost, SLA pressure, and load balance affect service quality, but they do not by themselves determine whether an action is physically deployable. The immediate reward is

$$r_t = r_t^{adm} - \beta_L \bar{L}_t - \beta_M \bar{C}_{mig,t} - \lambda_t \bar{V}_t^{sla} + r_t^{bal}, \tag{24}$$

where r_t^{adm} rewards useful admission, $\bar{L}_t$ is normalized latency, $\bar{C}_{mig,t}$ is normalized migration cost, $\bar{V}_t^{sla}$ is normalized SLA violation, and r_t^{bal} rewards balanced residual capacity.

The SLA penalty multiplier is adapted from recent violation behavior:

$$\lambda_{t+1} = \mathrm{clip}\left(\lambda_t + \eta_\lambda(\hat{V}_t^{sla} - \tau_{sla}), \lambda_{\min}, \lambda_{\max} \right), \tag{25}$$

where $\hat{V}_t^{sla}$ is the violation rate over a sliding window, τ_{sla} is the target violation tolerance, and η_λ is the update rate. This adaptive penalty regulates soft service-level pressure without weakening hard deployability constraints. In particular, MSD feasibility is never delegated to the reward; it is handled by the mask in Eq. (17).

5.5. Admission and Migration Semantics

The masked policy must define behavior when no feasible action exists. Under severe load, long service chains, or insufficient SID-depth headroom, $\mathcal{F}(s_t)$ may be empty. HARP then returns an explicit no-safe-action outcome:

$$\mathrm{Decision}(s_t) = \begin{cases} \arg\max_{a \in \mathcal{F}(s_t)} \pi_\theta^M(a \mid s_t), & \mathcal{F}(s_t) \neq \varnothing, \\ \texttt{NO_SAFE_ACTION}, & \mathcal{F}(s_t) = \varnothing. \end{cases} \tag{26}$$

This behavior is operationally important. In a saturated infrastructure, rejection is not necessarily failure; it is the correct outcome when every candidate action violates compute, memory, or MSD feasibility. The admission layer can then log the reason, expose it to the dashboard, and use it for capacity planning or retraining.

Migration is treated as one operation mode inside the same action space. A migration action is admissible only if the new placement and the resulting service path remain compute-, memory-, and MSD-feasible:

$$\phi_q^{new} \in \mathcal{F}_{place}(s_t) \quad \text{and} \quad \mathcal{S}_q(\phi_q^{new}) \in \mathcal{F}_{msd}(s_t). \tag{27}$$

The reward discourages unnecessary migration through C_{mig}, while latency and SLA terms encourage migration when the current placement degrades service quality. HARP therefore decides when migration is useful and where it should move the function; AIO-Mesh executes the migration through the make-before-break lifecycle described in Section 4.

5.6. Decision Procedure

Algorithm 2 summarizes the HARP decision procedure. The algorithm builds the graph-aware state representation, constructs the hard-feasibility mask, selects an action only from the feasible set, and returns the selected action together with a concrete feasible path descriptor. If no feasible action exists, it returns `NO_SAFE_ACTION`.

HARP is therefore not a packet-level path-programming algorithm. It is a topology-aware, hardware-constrained policy that ranks feasible orchestration actions. The controller realizes the chosen action through a concrete SRv6/P4 path, and the AIO-Mesh admission layer remains responsible for final validation before actuation.

6. Experiments and Evaluation

This section evaluates whether AIO-Mesh improves useful service-chain admission while preventing hardware-infeasible decisions from becoming admitted SRv6/P4 dataplane actions. We distinguish *unsafe candidates*, which fail

Algorithm 2 HARP decision procedure

Input: Request q, telemetry T_t, substrate graph $\mathcal{G}$, topology matrix $\mathcal{A}_{topo}$, cached path table $\mathcal{C}_K$
Output: Candidate action with feasible path descriptor, or `NO_SAFE_ACTION`

$\mathbf{X}_t, \mathbf{g}_t \leftarrow \textsc{BuildFeatures}(q, T_t, \mathcal{G})$
$\mathbf{H}^{(L)} \leftarrow \textsc{GATEncode}(\mathbf{X}_t, \mathcal{A}_{topo})$
$\mathbf{z}_t \leftarrow [\text{Pool}(\mathbf{H}^{(L)}) \| \mathbf{g}_t]$
$(\mathcal{F}(s_t), \mathcal{P}_t^{safe}) \leftarrow \textsc{BuildFeasibleSet}(q, T_t, \mathcal{G}, \mathcal{C}_K)$
if $\mathcal{F}(s_t) = \varnothing$ **then**
    return `NO_SAFE_ACTION`
end if
$\pi_\theta^M(\cdot \mid s_t) \leftarrow \textsc{MaskedPolicy}(\mathbf{z}_t, \mathcal{F}(s_t))$
$a_t \leftarrow \textsc{SelectAction}(\pi_\theta^M)$
$p_t \leftarrow \textsc{SelectLowestCostPath}(\mathcal{P}_t^{safe}(a_t))$
return (a_t, p_t)

hard feasibility checks before admission, from *admitted violations*, which are accepted actions that later violate data-plane constraints such as MSD. The evaluation is organized around five questions:

- **EQ1: Safe admission under stress.** Does HARP improve admission under heavy-tail, burst, and chaotic workloads while preserving hard feasibility?
- **EQ2: Topology sensitivity.** Does the policy remain effective across the Vietnam backbone, NSFNET, and GEANT2?
- **EQ3: Component contribution.** How do graph encoding, hard masking, adaptive penalties, and migration affect safe admission?
- **EQ4: Runtime deployability.** Can the AIOps runtime enforce admission before Kubernetes-based lifecycle control and SRv6/P4 steering?
- **EQ5: Operational interpretation.** Under saturation, does lower acceptance reflect weaker orchestration or correct safety-preserving rejection?

6.1. Experimental Setup

The algorithmic evaluation uses three substrate topologies: a 10-node Vietnam backbone, NSFNET, and GEANT2. The Vietnam topology represents a constrained national backbone with core and edge sites, while NSFNET and GEANT2 test whether the policy remains effective under different connectivity and path-diversity profiles. Core nodes are assigned higher compute capacity and larger MSD limits, whereas edge nodes have lower resource and parser-depth headroom.

The workload consists of SFC requests with source–destination pairs, ordered VNF chains, resource demands, service classes, latency targets, and SRv6 path constraints. The chains include functions such as firewalling, intrusion detection, deep inspection, load balancing, and routing; AI-enabled traffic-analysis functions are motivated by prior operational security-function studies [48, 49, 50]. We use three stress profiles. **Elephant stress** contains large requests that consume disproportionate compute and path-depth resources. **Burst surge** compresses many arrivals into a short interval. **Chaos stress** combines heavy-tail demand, heterogeneous service classes, and diverse source–destination pairs. The workloads intentionally exceed comfortable capacity, so low raw acceptance is expected in the hardest cases. Each simulation result is averaged over 5 independent runs with different random seeds, and the reported deviations are standard deviations. Unless otherwise stated, the candidate-path cache uses $K = 5$ shortest simple paths per source–anchor–destination tuple. This value provides multiple SRv6-realizable alternatives for each compact orchestration action while keeping feasibility-mask construction lightweight for the evaluated Vietnam, NSFNET, and GEANT2 topologies. The load-sweep experiment normalizes arrival intensity against the nominal workload rate and varies the offered-load multiplier from $0.2\times$ to $1.5\times$. All baselines are evaluated under the same topology, workload generator, resource-capacity settings, and MSD-admission accounting.

The hardware-in-the-loop evaluation uses the 10-node Vietnam backbone implemented with MicroK8s, Tekton, Mininet, BMv2 P4 switches, SRv6 steering, FastAPI-based orchestration, and a telemetry-driven dashboard. The testbed evaluates whether unsafe candidates are rejected before data-plane actuation rather than merely visualizing policy decisions.

6.2. Baselines, Ablation Variants, and Metrics

We compare HARP with heuristic, learning-based, and runtime variants following recent DRL, GCN-based, and migration-aware SFC orchestration studies [13, 14, 18, 51, 20]. **Greedy** is a short-horizon placement–routing baseline that uses local feasibility and immediate cost, but does not account for future fragmentation, SID-stack pressure, or migration effects. **Decoupled AI (DAI)** uses learned placement or routing behavior without the joint graph-aware masking structure used by HARP. **SRv6-aware feasible heuristic (SAF-H)** filters candidate placements and paths by CPU, memory, and SRv6 MSD feasibility before admission, then selects the feasible action with the lowest weighted cost. SAF-H is a stronger deterministic baseline than Greedy, but remains myopic and non-adaptive. In the hardware-in-the-loop testbed, we additionally evaluate a safety-protected **Heuristic runtime** and a **Hybrid runtime**; the latter uses the heuristic path under stable load and invokes HARP under stress, alert, or migration pressure.

We use two comparison modes. In the simulation study, Greedy, DAI, SAF-H, and HARP are evaluated under the same topology, workload generator, resource-capacity settings, MSD profile, and metric definitions, so that safe

acceptance and pre-admission infeasible pressure are measured consistently. In the hardware-in-the-loop testbed, HARP, Hybrid runtime, and Heuristic runtime are evaluated as safety-protected runtime variants because their decisions pass through the AIO-Mesh admission boundary before actuation. Greedy and DAI are evaluated as unprotected stress baselines; their role is to expose the data-plane consequence of bypassing hardware-safe admission checks, rather than to represent protected AIO-Mesh runtime behavior.

For ablation, **PPO-MLP** replaces the graph-attention encoder with a flat MLP. **HARP w/o masking** disables policy-side hard masking while retaining runtime admission. **HARP w/o mask + unchecked** removes both policy-side masking and runtime admission checks, and is used only as a diagnostic unsafe variant. **HARP-SoftMSD** treats MSD feasibility as a reward penalty. **HARP w/o adaptive penalty** uses fixed soft-constraint weights, and **HARP w/o migration** disables proactive migration actions. These variants isolate topology awareness, hard deployability enforcement, runtime safety validation, adaptive soft-constraint control, and migration behavior.

We report acceptance rate, safe acceptance rate, pre-admission hard-infeasible candidate rate, admitted MSD violation rate, average latency, SLA violation rate, queueing delay, and reward. Safe acceptance is the primary admission metric because it counts only requests satisfying compute, memory, and MSD feasibility. Pre-admission hard-infeasible candidate rate measures how strongly the workload pushes the system outside the feasible region before admission, while admitted MSD violation rate measures actual data-plane safety failure after admission.

6.3. Hardware-Safe Admission under Workload Saturation

To test behavior under progressive saturation, we vary the request arrival rate on the Vietnam topology. As shown in Figure 2, safe acceptance decreases as the offered load increases, while simulation-side hard-admission filtering prevents infeasible candidates from becoming admitted MSD violations.

6.4. Simulation Results across Distributed Topologies

Table 2 summarizes safe acceptance across topologies and stress workloads. HARP consistently improves over Greedy and DAI. SAF-H is competitive because it also enforces CPU, memory, and MSD feasibility before admission; however, it remains myopic and lacks the topology-aware adaptation and migration behavior of HARP. On Vietnam, HARP reaches 17.89%−18.74% safe acceptance, compared with approximately 4% for Greedy and DAI. On NSFNET, HARP reaches 29.41%−31.51%, roughly four times the Greedy rate. GEANT2 provides richer path diversity, where HARP reaches 44.73%−49.52%.

Table 3 provides a more detailed comparison for the Burst-surge workload on Vietnam and GEANT2. On the constrained Vietnam backbone, HARP improves admission and latency over all baselines while preserving zero

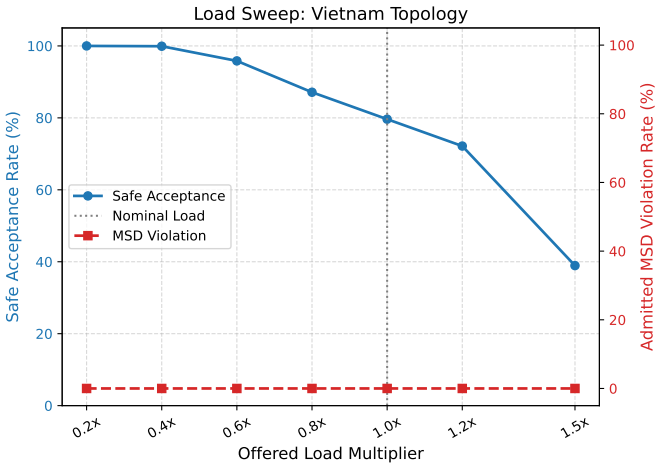


Figure 2: Load-sweep simulation on the Vietnam backbone under varying offered-load multipliers. The figure reports safe acceptance and admitted MSD violation behavior under simulation-side hard-admission filtering. Unsafe candidates are rejected before actuation, resulting in zero admitted MSD violations across the tested load range.

Table 2: Stress simulation summary: safe acceptance rate across topologies and workloads (%).

Scenario	Greedy	DAI	SAF-H	HARP
Vietnam backbone topology				
Elephant	4.07 ± 0.20	4.40 ± 0.20	16.27 ± 0.59	**17.89 ± 0.67**
Burst surge	4.29 ± 0.15	4.43 ± 0.80	17.47 ± 0.43	**18.74 ± 0.51**
Chaos	4.07 ± 0.20	4.40 ± 0.20	16.27 ± 0.59	**17.89 ± 0.67**
NSFNET topology				
Elephant	6.81 ± 0.67	6.41 ± 0.46	**29.61 ± 0.36**	29.41 ± 0.47
Burst surge	7.66 ± 0.45	6.45 ± 1.69	**32.32 ± 0.50**	31.51 ± 0.81
Chaos	6.81 ± 0.67	6.41 ± 0.46	**29.61 ± 0.36**	29.41 ± 0.47
GEANT2 topology				
Elephant	8.47 ± 0.91	8.14 ± 1.72	43.82 ± 0.86	**44.73 ± 0.80**
Burst surge	8.67 ± 0.61	9.21 ± 1.59	48.26 ± 0.64	**49.52 ± 0.48**
Chaos	8.47 ± 0.91	8.14 ± 1.72	43.82 ± 0.86	**44.73 ± 0.80**

SLA violations. On GEANT2, HARP achieves the highest admission and reward. SAF-H shows lower pre-admission MSD-infeasible pressure on GEANT2 because it is more conservative, but it admits fewer useful requests. This confirms that unsafe-candidate pressure should be interpreted together with safe acceptance: the goal is not merely to avoid considering infeasible candidates, but to admit more deployable requests without creating admitted data-plane violations.

Overall, the simulation results show that HARP is not simply rejecting more aggressively. It selects more useful actions from the feasible region, especially when topology diversity provides alternative placement–routing combinations. Latency should be interpreted jointly with acceptance: a baseline may appear fast by admitting only a small and easy subset of requests, whereas HARP admits a larger deployable set under stress.

Table 3: Burst-surge simulation results on the Vietnam backbone and GEANT2 topologies.

Method	Adm.	Pre-adm. MSD	SLA	Lat.	Queue	Reward
Vietnam backbone simulation						
Traditional Greedy	4.3	95.7	0.0	2.5	0.07	−14.0
Decoupled AI	4.4	95.6	0.0	1.2	0.06	−14.4
SAF-H	17.5	82.5	0.0	0.4	0.08	0.0
HARP	18.7	84.5	0.0	0.2	0.10	1.4
GEANT2 simulation						
Traditional Greedy	8.7	91.3	0.0	3.6	0.07	−8.2
Decoupled AI	9.2	90.8	0.2	2.1	0.06	−8.4
SAF-H	48.3	51.7	0.2	1.0	0.10	35.6
HARP	49.5	64.5	0.0	0.6	0.16	35.8

Table 4: Ablation results on the Vietnam topology (%).

Var.	Safe acc.	Pre-adm. MSD	Adm. MSD	SLA	Lat. ms
Uniform					
Full	**18.90 ± 0.45**	81.05 ± 0.52	**0.00 ± 0.00**	**0.00 ± 0.00**	1.44 ± 0.09
MLP	17.47 ± 0.51	82.51 ± 0.50	**0.00 ± 0.00**	**0.00 ± 0.00**	3.43 ± 0.15
NoMask	18.59 ± 0.50	81.11 ± 0.62	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.78 ± 0.06**
NoMask+U	4.99 ± 0.54	91.01 ± 4.33	23.56 ± 1.56	**0.00 ± 0.00**	4.37 ± 0.12
SoftMSD	1.39 ± 0.22	97.39 ± 0.73	34.53 ± 0.80	0.03 ± 0.04	2.53 ± 0.16
NoAdapt	16.48 ± 0.40	83.48 ± 0.42	**0.00 ± 0.00**	**0.00 ± 0.00**	1.73 ± 0.43
Bursty					
Full	**18.74 ± 0.51**	81.25 ± 0.53	**0.00 ± 0.00**	**0.00 ± 0.00**	0.87 ± 0.13
MLP	17.72 ± 0.38	82.28 ± 0.38	**0.00 ± 0.00**	**0.00 ± 0.00**	3.74 ± 0.08
NoMask	13.77 ± 1.47	86.19 ± 1.46	**0.00 ± 0.00**	**0.00 ± 0.00**	**0.26 ± 0.05**
NoMask+U	4.41 ± 0.34	93.59 ± 1.17	31.78 ± 1.06	0.01 ± 0.01	4.08 ± 0.10
SoftMSD	1.58 ± 0.14	98.29 ± 0.23	34.38 ± 1.06	**0.00 ± 0.00**	2.84 ± 0.10
NoAdapt	16.51 ± 0.75	83.49 ± 0.75	**0.00 ± 0.00**	**0.00 ± 0.00**	0.93 ± 0.18
Heavy-Tail					
Full	**17.89 ± 0.67**	81.97 ± 0.65	**0.00 ± 0.00**	**0.00 ± 0.00**	1.88 ± 0.16
MLP	16.57 ± 0.53	83.43 ± 0.53	**0.00 ± 0.00**	**0.00 ± 0.00**	3.29 ± 0.19
NoMask	17.15 ± 0.74	82.26 ± 0.70	**0.00 ± 0.00**	**0.00 ± 0.00**	**1.04 ± 0.11**
NoMask+U	3.89 ± 0.31	94.04 ± 1.25	23.22 ± 0.62	0.07 ± 0.05	4.68 ± 0.17
SoftMSD	1.12 ± 0.15	98.19 ± 0.77	33.57 ± 0.41	0.03 ± 0.03	2.25 ± 0.03
NoAdapt	16.06 ± 0.43	83.93 ± 0.43	**0.00 ± 0.00**	**0.00 ± 0.00**	1.71 ± 0.26

6.5. Ablation Study

The ablation study isolates the main HARP design choices on the Vietnam topology, where compute and MSD headroom are limited. Table 4 shows that full HARP obtains the highest safe acceptance across Uniform, Bursty, and Heavy-tail workloads while maintaining 0.0% admitted MSD violations.

The ablation confirms four points. First, replacing graph attention with a flat MLP reduces safe acceptance and increases latency, showing that topology information is important for placement–routing decisions. Second, disabling policy-side masking while keeping runtime admission can remain safe on the small Vietnam topology, but it loses admission quality under bursty load; the unchecked variant exposes the true risk by producing 23.22%−31.78% admitted MSD violations. Third, treating MSD as a reward penalty is ineffective: SoftMSD collapses safe acceptance to about 1%−2% and still admits many MSD-violating requests. Fourth, removing adaptive penalties remains safe but reduces safe acceptance, indicating that adaptive soft-constraint control improves service quality without weakening hardware safety.

Table 5: Hardware-in-the-loop testbed results: acceptance rate / admitted MSD violation rate.

Algorithm	Normal	Elephant	Topo stress	DDoS MBB	Chaos
HARP	100.0 / 0.0	71.0 / 0.0	21.2 / 0.0	23.0 / 0.0	10.0 / 0.0
Hybrid runtime	100.0 / 0.0	75.3 / 0.0	21.2 / 0.0	24.8 / 0.0	8.3 / 0.0
Heuristic	100.0 / 0.0	65.6 / 0.0	20.3 / 0.0	22.1 / 0.0	7.9 / 0.0
Greedy	100.0 / 0.0	59.1 / 19.4	21.2 / 78.8	25.7 / 69.0	7.9 / 79.2
DAI	94.1 / 5.9	52.7 / 22.6	9.3 / 90.7	18.6 / 77.9	5.8 / 81.3

Figure 3 extends this component analysis to NSFNET and GEANT2. The figure shows that full HARP remains on or near the Pareto frontier when safe acceptance is evaluated jointly with SLA violation, latency, and switching behavior. This is important because a variant may appear competitive on acceptance alone while being dominated once routing quality or action stability is considered. The large-topology results therefore support the same conclusion as the Vietnam ablation: HARP's full design is not only safer than unchecked variants, but also provides a better multi-objective operating point than flat, unmasked, or fixed-penalty alternatives.

6.6. Hardware-in-the-Loop Testbed Validation

The hardware-in-the-loop testbed evaluates whether the AIO-Mesh runtime can prevent unsafe policy outputs from becoming admitted SRv6/P4 actions. As shown in Figure 4, the testbed integrates the AIO-Mesh FastAPI backend, HARP policy engine, smart admission controller, MicroK8s/Tekton-based VNF lifecycle control, a Mininet-based Vietnam topology, BMv2 P4 switches, SRv6 steering, and MSD-drop counters. We evaluate five scenarios: Normal load, Elephant heavy-tail, Topology and physics stress, DDoS with make-before-break triggering, and Chaos.

The comparison separates protected runtime behavior from unprotected decision making. HARP, Hybrid runtime, and Heuristic runtime pass all candidate actions through the AIO-Mesh safety and admission boundary; infeasible decisions are converted into explicit `NO_SAFE_ACTION` outcomes before actuation. Greedy and DAI are executed as unprotected stress baselines to expose the consequence of bypassing hardware-safe admission checks.

Table 5 reports acceptance rate and admitted MSD violation rate. HARP and the Hybrid runtime maintain 0.0% admitted MSD violations in all scenarios. Greedy and DAI can appear competitive in raw acceptance, but they produce high admitted MSD violation rates under stress. For example, in the DDoS make-before-break scenario, Greedy accepts 25.7% of requests but incurs 69.0% admitted MSD violations, whereas HARP accepts 23.0% with zero admitted violations. Under Chaos, HARP accepts only 10.0%, but this reflects correct safety-preserving rejection under saturation rather than orchestration failure.

These results confirm the separation between simulation-side unsafe candidate pressure and runtime data-plane safety. Simulation quantifies how often workloads push

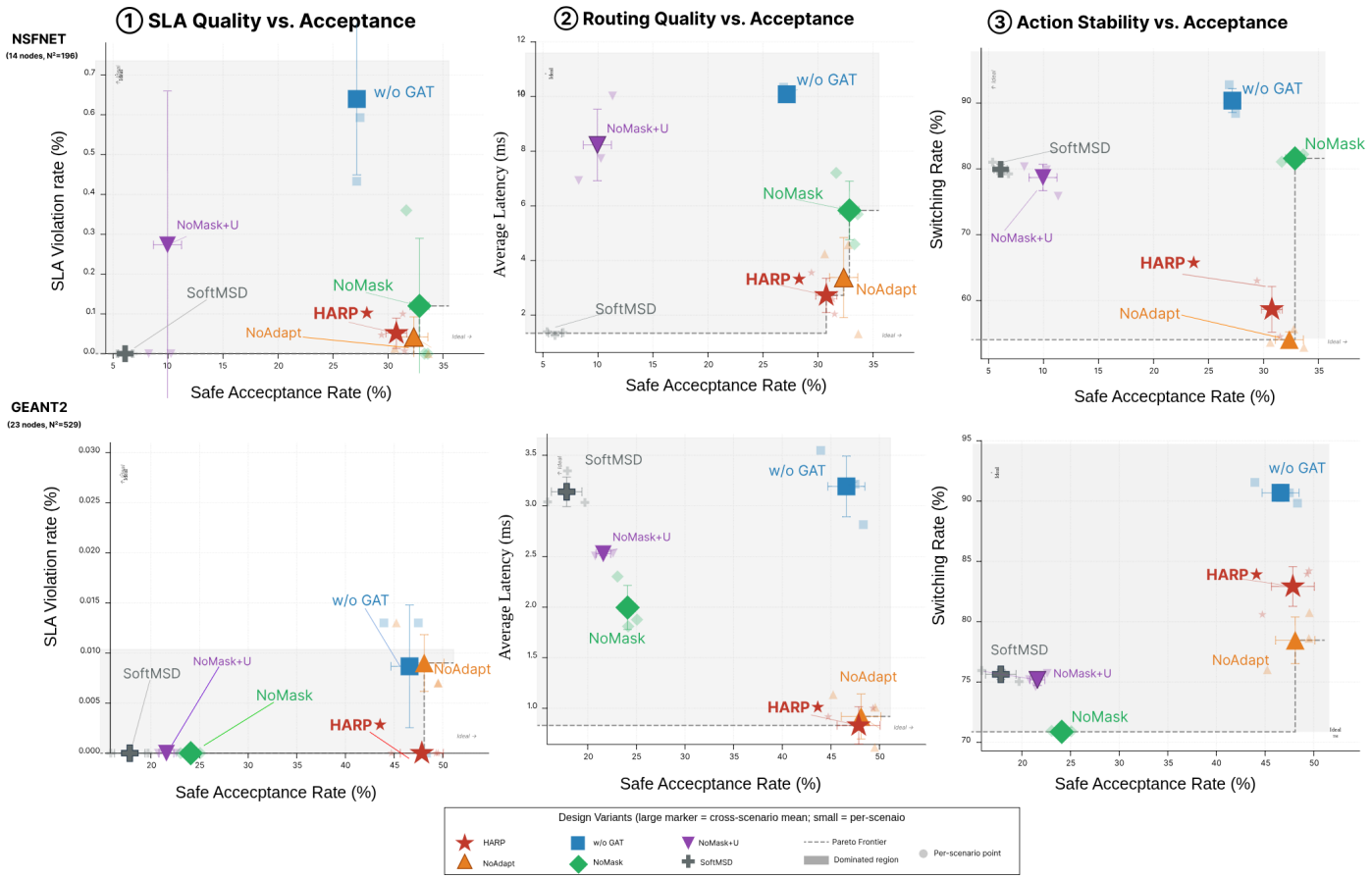


Figure 3: Architectural design-space trade-offs on NSFNET and GEANT2. Each column shows a pairwise trade-off between safe acceptance rate (↑) and a cost metric (↓). Large markers denote cross-scenario means, while small markers denote per-scenario values. The dashed line marks the Pareto frontier, and the shaded region contains dominated design points. Full HARP is Pareto-optimal or near-optimal across the six trade-off views.

the policy toward infeasible regions, while the hardware-in-the-loop testbed verifies that protected AIO-Mesh variants reject such candidates before they become admitted SRv6/P4 data-plane actions.

6.7. Operational Interpretation, Overhead, and Limitations

The evaluation supports three operational conclusions. First, safe acceptance is more informative than raw acceptance for SRv6/P4 service-chain orchestration. A request accepted with an MSD violation is not deployable. HARP and the Hybrid runtime sometimes accept fewer requests than Greedy, but their accepted actions remain hardware-safe. Second, the runtime architecture is part of the contribution: AIO-Mesh converts a graph RL policy into an operational control loop by combining telemetry, inference, hard-feasibility masking, admission, lifecycle actuation, programmable steering, and feedback. Third, make-before-break migration is validated as a safety-preserving control sequence rather than as a packet-level zero-downtime guarantee. In the DDoS make-before-break scenario, the runtime creates and verifies the replacement VNF, installs SRv6-safe steering, and preserves zero admitted MSD violations; strict zero-downtime would require TCP or packet-capture continuity evidence.

The policy-side overhead is practical for the evaluated topologies. For a substrate with N nodes, E edges, K attention heads, and hidden dimension F, one graph-attention layer costs

$$\mathcal{O}(NF^2K + EFK). \tag{28}$$

Because N and E describe infrastructure sites and links rather than individual flows, this cost is modest for backbone-scale substrates. Cached candidate paths and SID-depth metadata further reduce feasibility-mask construction to current resource and header-depth checks over the candidate action set. Larger delays are expected from actuation rather than policy inference: Kubernetes/Tekton deployment, readiness checks, SRv6/P4 rule installation, and controller confirmation depend on the runtime backend and programmable target.

The current prototype has three limitations. BMv2 validates parser-depth logic and control-plane behavior, but not ASIC line-rate forwarding. The workloads are synthetic and stress-oriented; production traces would strengthen external validity. Finally, the forecasting adapter is used as an operational trigger rather than as a separately evaluated

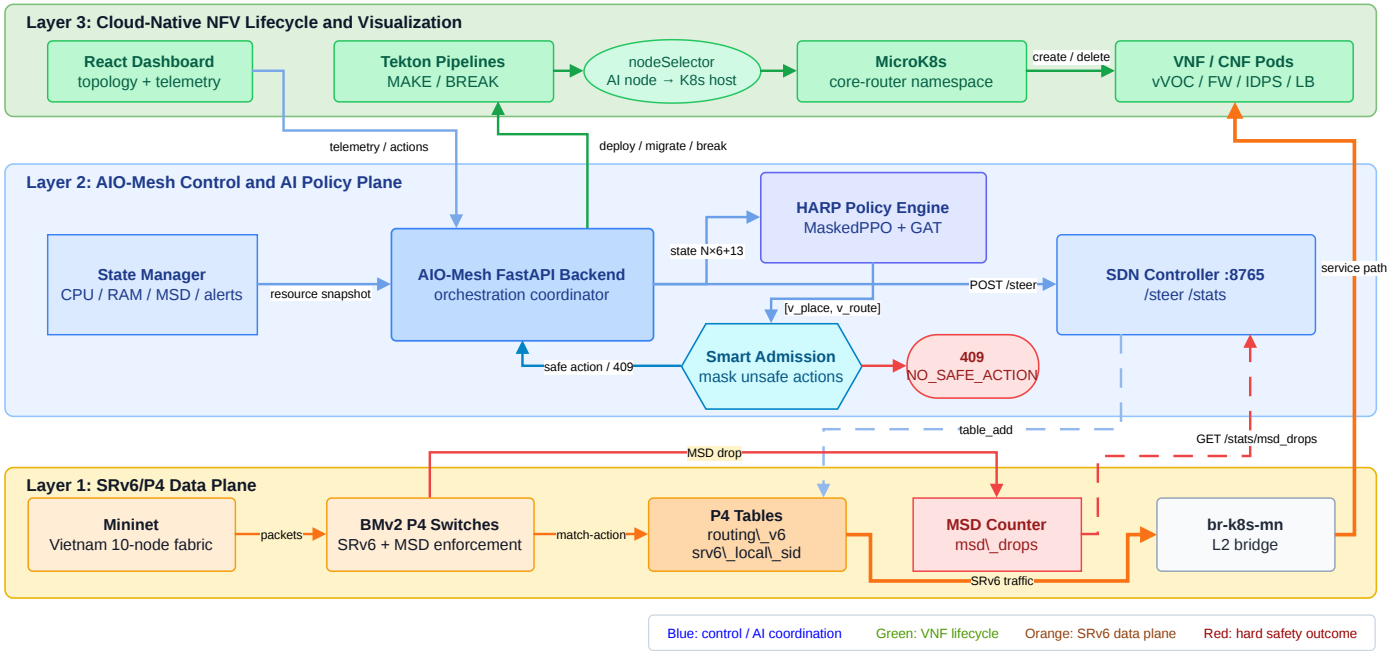


Figure 4: Hardware-in-the-loop testbed used to validate the AIO-Mesh control loop with MicroK8s, Tekton, Mininet, BMv2 P4 switches, SRv6 steering, and runtime admission control.

prediction model. These limitations do not weaken the central result: AIO-Mesh embeds a learned AIOps policy into a cloud-native SRv6/P4 orchestration loop while preventing hardware-infeasible actions from becoming admitted data-plane behavior.

7. Conclusion

This paper presented **AIO-Mesh**, a hardware-safe AIOps orchestration framework for cloud-native service function chains over programmable SRv6/P4 data planes. The central motivation is that learned orchestration policies cannot be treated as offline optimizers when their outputs directly affect service placement, traffic steering, migration, and programmable forwarding behavior. In such environments, a logically efficient service path may still be non-deployable if its SID stack exceeds the parser-depth capability of the forwarding profile.

At the core of AIO-Mesh, we introduced **HARP**, a hardware-aware reinforcement policy that combines graph-attention topology encoding, masked policy optimization, adaptive penalties, and smart admission control. The key design principle is to separate hard deployability constraints from soft operational objectives. CPU, memory, and SRv6 MSD feasibility are enforced through action masking and admission control before actuation, whereas latency, SLA pressure, load balance, and migration cost are optimized through the reward function.

The evaluation shows that HARP improves safe admission under stress-oriented workloads across the Vietnam backbone, NSFNET, and GEANT2. The hardware-in-the-loop testbed further demonstrates that AIO-Mesh can embed the learned policy into an operational control loop with telemetry-driven state construction, Kubernetes/Tekton-based VNF lifecycle control, SRv6/P4 steering, and runtime feedback, while maintaining zero admitted MSD violations. Under saturation, AIO-Mesh converts infeasible candidates into explicit `NO_SAFE_ACTION` outcomes rather than allowing them to become data-plane failures.

The current prototype has several limitations. The programmable data plane is validated with BMv2 rather than production ASIC or smartNIC/DPU targets, and the workloads are synthetic stress scenarios rather than production traces. Make-before-break migration is implemented as a safety-preserving control sequence, but strict zero-downtime validation requires packet-level continuity evidence. Future work will extend AIO-Mesh toward production programmable hardware, fuller MLOps lifecycle management, cross-topology transfer, and packet-level validation of migration continuity.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the Ministry of Science and Technology of Vietnam under the National Science and Technology Program, project code KC-4.0-56/19-30.

Data and Code Availability

The simulation scenarios, topology descriptions, workload configurations, testbed scripts, Mininet/BMv2 setups, SRv6/P4 steering scripts, Tekton pipeline definitions, and evaluation logs used in this study are publicly available at: https://github.com/uet-dis/AIO-Mesh/.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the authors used generative AI and AI-assisted technologies to refine the scientific prose, improve readability, and polish LaTeX formatting. After using these technologies, the authors thoroughly reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT Authorship Contribution Statement

Hanh P. Du: Conceptualization, Methodology, Formal analysis, Validation, Writing – original draft. **Hung D. Hoang**: Software, Investigation, Testbed implementation, Visualization, Validation, Writing – original draft. **Thu H. Le**: Software, Data curation, Experimental evaluation, Visualization, Writing – review & editing. **Tuan X. Le**: Software, Testbed integration, Programmable dataplane configuration, Validation, Writing – review & editing. **Giang L. Nguyen**: Formal analysis, Methodology, Validation, Writing – review & editing. **Hoa N. Nguyen**: Conceptualization, Methodology, Supervision, Project administration, Funding acquisition, Writing – review & editing.

References

[1] H. V. Vo, H. N. Nguyen, T. N. Nguyen, H. P. Du, Sdaid: Towards a hybrid signature and deep analysis-based intrusion detection method, in: GLOBECOM 2022 - 2022 IEEE Global Communications Conference, 2022, pp. 2615–2620. doi:10.1109/GLOBECOM48099.2022.10001582.

[2] D. Kreuzberger, N. Kühl, S. Hirschl, Machine learning operations (MLOps): Overview, definition, and architecture, IEEE Access 11 (2023) 31866–31879. doi:10.1109/ACCESS.2023.3262138.

[3] J. Díaz-de Arcaya, R. Miñón, A. I. Torre-Bastida, J. Del Ser, A. Almeida, A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey, ACM Computing Surveys 56 (4) (2024) 1–30. doi:10.1145/3625289.

[4] B. Eken, S. Pallewatta, N. K. Tran, A. Tosun, M. A. Babar, A multivocal review of MLOps practices, challenges and open issues, ACM Computing Surveys 58 (2) (2025) 1–40. doi:10.1145/3747346.

[5] F. Bayram, B. S. Ahmed, Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach, ACM Computing Surveys 57 (5) (2025) 1–41. doi:10.1145/3708497.

[6] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P. S. Yu, Y. Li, A survey of AIOps in the era of large language models, ACM Computing Surveys 58 (2) (2025) 1–35. doi:10.1145/3746635.

[7] R. Eramo, B. Said, M. Oriol, H. Bruneliere, S. Morales, An architecture for model-based and intelligent automation in DevOps, Journal of Systems and Software 217 (2024) 112180. doi:10.1016/j.jss.2024.112180.

[8] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, R. Shakir, Segment routing architecture, RFC 8402, RFC Editor (2018). doi:10.17487/RFC8402.

[9] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, Z. Li, Segment routing over IPv6 (SRv6) network programming, RFC 8986, RFC Editor (2021). doi:10.17487/RFC8986.

[10] P. Psenak, C. Filsfils, A. Bashandy, B. Decraene, Z. Hu, IS-IS extensions to support segment routing over the IPv6 dataplane, RFC 9352, RFC Editor (2023). doi:10.17487/RFC9352.

[11] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, D. Walker, P4: Programming protocol-independent packet processors, ACM SIGCOMM Computer Communication Review 44 (3) (2014) 87–95. doi:10.1145/2656877.2656890.

[12] T. Wang, Q. Fan, X. Li, X. Zhang, Q. Xiong, S. Fu, M. Gao, DRL-SFCP: Adaptive service function chains placement with deep reinforcement learning, in: Proceedings of the IEEE International Conference on Communications (ICC), IEEE, 2021, pp. 1–6. doi:10.1109/ICC42927.2021.9500964.

[13] D. Xiao, J. A. Zhang, X. Liu, Y. Qu, W. Ni, R. P. Liu, A two-stage GCN-based deep reinforcement learning framework for SFC embedding in multi-datacenter networks, IEEE Transactions on Network and Service Management 20 (4) (2023) 4297–4312. doi:10.1109/TNSM.2023.3284293.

[14] N. Liu, J. Zhang, Service function chain embedding meets machine learning: Deep reinforcement learning approach, IEEE Transactions on Network and Service Management 21 (3) (2024) 3465–3481. doi:10.1109/TNSM.2024.3353808.

[15] W. Taktak, M. Escheikh, K. Barkaoui, DRL approach for online user-centric QoS-aware SFC embedding with

dynamic VNF placement, Computer Networks 251 (2024) 110637. doi:10.1016/j.comnet.2024.110637.

[16] Y. Wu, H. Hu, Z. Zhang, DRL-based fast joint mapping approach for SFC deployment, Electronics 14 (12) (2025) 2408. doi:10.3390/electronics14122408.

[17] D. Zhai, X. Meng, Z. Yu, H. Hu, Y. Liang, A migration method for service function chain based on failure prediction, Computer Networks 222 (2023) 109554. doi:10.1016/j.comnet.2022.109554.

[18] R. A. Addad, D. L. C. Dutra, T. Taleb, H. Flinck, AI-based network-aware service function chain migration in 5G and beyond networks, IEEE Transactions on Network and Service Management 19 (1) (2022) 472–484. doi:10.1109/TNSM.2021.3074618.

[19] Y. Qin, W. Zhang, H. Chen, J. Wang, C. Feng, J. Wang, Service function chain migration with the long-term budget in dynamic networks, Computer Networks 223 (2023) 109563. doi:10.1016/j.comnet.2023.109563.

[20] S. N. Afrasiabi, A. Ebrahimzadeh, N. Promwongsa, C. Mouradian, W. Li, Á. Recse, R. Szabó, R. H. Glitho, Cost-efficient cluster migration of VNFs for service function chain embedding, IEEE Transactions on Network and Service Management 21 (1) (2024) 979–993. doi:10.1109/TNSM.2023.3287757.

[21] Z. Zhang, C. Wang, Service function chain migration: A survey, Computers 14 (6) (2025) 203. doi:10.3390/computers14060203.

[22] H. Hu, Z. Liu, F. Wu, RL-PMO: A reinforcement learning-based optimization algorithm for parallel SFC migration, Sensors 26 (1) (2026) 242. doi:10.3390/s26010242.

[23] C. Qiu, B. Ren, L. Luo, D. Guo, SFCPlanner: An online SFC planning approach with SRv6 flow steering, IEEE Transactions on Network and Service Management 21 (4) (2024) 4625–4638. doi:10.1109/TNSM.2024.3392945.

[24] Z. Ding, S. Wang, C. Jiang, Kubernetes-oriented microservice placement with dynamic resource allocation, IEEE Transactions on Cloud Computing 11 (2) (2023) 1777–1793. doi:10.1109/TCC.2022.3161900.

[25] Y. Mao, V. Sharma, W. Zheng, L. Cheng, Q. Guan, A. Li, FlowCon: Elastic resource management for deep learning applications in a container cluster, IEEE Transactions on Cloud Computing 11 (3) (2023) 2204–2216. doi:10.1109/TCC.2022.3194128.

[26] X. Chen, L. Yang, Z. Chen, G. Min, X. Zheng, C. Rong, Resource allocation with workload-time windows for cloud-based software services: A deep reinforcement learning approach, IEEE Transactions on Cloud Computing 11 (2) (2023) 1871–1885. doi:10.1109/TCC.2022.3169157.

[27] J. A. Hortelano, Deep reinforcement learning for intelligent resource allocation in edge and cloud environments, Future Generation Computer Systems 148 (2023) 52–67. doi:10.1016/j.future.2023.01.009.

[28] C. Chen, M. Herrera, G. Zheng, L. Xia, Z. Ling, J. Wang, Cross-edge orchestration of serverless functions with probabilistic caching, IEEE Transactions on Services Computing 17 (3) (2024) 678–691. doi:10.1109/TSC.2024.3399651.

[29] S. Sedghani, D. Filippini, D. Ardagna, SPACE4AI-D: A design-time tool for AI applications resource selection in computing continua, IEEE Transactions on Services Computing (2024). doi:10.1109/TSC.2024.3479935.

[30] J. Li, Z. Zhu, QoS-aware management reconfiguration of vNF service trees with heterogeneous NFV platforms, IEEE Transactions on Network and Service Management 20 (2) (2023) 2013–2024. doi:10.1109/TNSM.2022.3220229.

[31] X. Pei, S. Guo, Y. Hu, Z. Li, Q. Yao, D. Li, J. Pei, Y. Dong, Graph pointer network assisted deep reinforcement learning for virtualized network embedding, IEEE Transactions on Green Communications and Networking 9 (4) (2025) 2222–2235. doi:10.1109/TGCN.2025.3548140.

[32] A. Stockmayer, S. Hinselmann, M. Häberle, M. Menth, Service function chaining based on segment routing using P4 and SR-IOV (P4-SFC), in: High Performance Computing, Vol. 12321 of Lecture Notes in Computer Science, Springer, Cham, 2020, pp. 297–309. doi:10.1007/978-3-030-59851-8_19.

[33] D. Zhang, X. Chen, Q. Huang, X. Hong, C. Wu, H. Zhou, Y. Yang, H. Liu, Y. Chen, P4SC: A high performance and flexible framework for service function chain, IEEE Access 7 (2019) 160982–160997. doi:10.1109/ACCESS.2019.2950446.

[34] X. Yan, N. L. S. da Fonseca, Z. Zhu, Self-adaptive SRv6-INT-driven system adjustment in runtime for reliable service function chaining, IEEE Transactions on Network and Service Management 21 (5) (2024) 4962–4973. doi:10.1109/TNSM.2024.3404461.

[35] J. Li, Y. Ma, Z. Zhu, st-SFC: Optimizing dynamic deployment of stateful SFCs on P4-based PDP switches, IEEE Transactions on Network and Service Management (2024). doi:10.1109/TNSM.2024.3439515.

[36] Y. Ma, J. Li, X. Xu, N. L. S. da Fonseca, Z. Zhu, SF-Cache: Hybrid NF synthesization in runtime with rule-caching in programmable switches, IEEE Transactions

on Network and Service Management 21 (4) (2024) 4613–4624. doi:10.1109/TNSM.2024.3390140.

[37] J. Halpern, C. Pignataro, Service function chaining (SFC) architecture, RFC 7665, RFC Editor (2015). doi:10.17487/RFC7665.

[38] T. N. Nguyen, T. V. Le, M. V. Nguyen, H. N. Nguyen, S. Vu, Optimizing resource allocation and vnf embedding in ran slicing, IEEE Transactions on Network and Service Management 21 (2) (2024) 2187–2199. doi:10.1109/TNSM.2023.3319309.

[39] H. Qu, K. Wang, J. Zhao, Reliable service function chain deployment method based on deep reinforcement learning, Sensors 21 (8) (2021) 2733. doi:10.3390/s21082733.

[40] Y. Yang, L. He, J. Zhou, X. Shi, J. Cao, Y. Liu, P4runpro: Enabling runtime programmability for RMT programmable switches, in: Proceedings of the ACM SIGCOMM 2024 Conference, ACM, 2024, pp. 921–937. doi:10.1145/3651890.3672230.

[41] F. Ruffy, J. Liu, P. Kotikalapudi, V. Havel, H. Tavante, R. Sherwood, V. Dubina, V. Peschanenko, A. Sivaraman, N. Foster, P4Testgen: An extensible test oracle for P4, in: Proceedings of the ACM SIGCOMM 2023 Conference, ACM, 2023, pp. 136–151. doi:10.1145/3603269.3604834.

[42] Y. Zhao, W. Liu, F. Dong, T. Yang, Y. Li, K. Yang, Z. Liu, Z. Jia, Y. Yang, P4LRU: Towards an LRU cache entirely in programmable data plane, in: Proceedings of the ACM SIGCOMM 2023 Conference, ACM, 2023, pp. 788–801. doi:10.1145/3603269.3604813.

[43] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, foundational PPO reference; no formal DOI-bearing proceedings version is available (2017). arXiv:1707.06347.

[44] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations (ICLR), 2018.
URL https://iclr.cc/virtual/2018/poster/299

[45] S. Gu, L. Yang, Y. Du, G. Chen, F. Walter, J. Wang, A. Knoll, A review of safe reinforcement learning: Methods, theory and applications, IEEE Transactions on Pattern Analysis and Machine Intelligence (2024). doi:10.1109/TPAMI.2024.3457538.

[46] Y. J. Ma, A. Shen, O. Bastani, D. Jayaraman, Conservative and adaptive penalty for model-based safe reinforcement learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 7573–7581. doi:10.1609/aaai.v36i7.20720.

[47] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, in: Proceedings of the Thirty-Fifth International Florida Artificial Intelligence Research Society Conference, Vol. 35, 2022. doi:10.32473/flairs.v35i.130584.

[48] H. V. Vo, H. P. Du, H. N. Nguyen, Ai-powered intrusion detection in large-scale traffic networks based on flow sensing strategy and parallel deep analysis, Journal of Network and Computer Applications 220 (2023) 103735. doi:10.1016/j.jnca.2023.103735.

[49] H. V. Vo, H. P. Du, H. N. Nguyen, APELID: Enhancing real-time intrusion detection with augmented wgan and parallel ensemble learning, Computers & Security 136 (2024) 103567. doi:10.1016/j.cose.2023.103567.

[50] T. T. Nguyen, H. T. Nguyen, M. D. Nguyen, H. H. Le, H. N. Nguyen, CSFS: Enhancing industrial threat detection performance with covariate shift adaptation and shap-powered feature selection, in: 2025 17th International Conference on Knowledge and System Engineering (KSE), 2025, pp. 1–6. doi:10.1109/KSE68178.2025.11309537.

[51] M. A. Tariq, M. U. Farooq, M. Zeeshan, A. Hassan, A. Akhunzada, FIPAM: Fuzzy inference based placement and migration approach for NFV-based IoTs, IEEE Transactions on Network and Service Management 19 (4) (2022) 4298–4309. doi:10.1109/TNSM.2022.3145658.